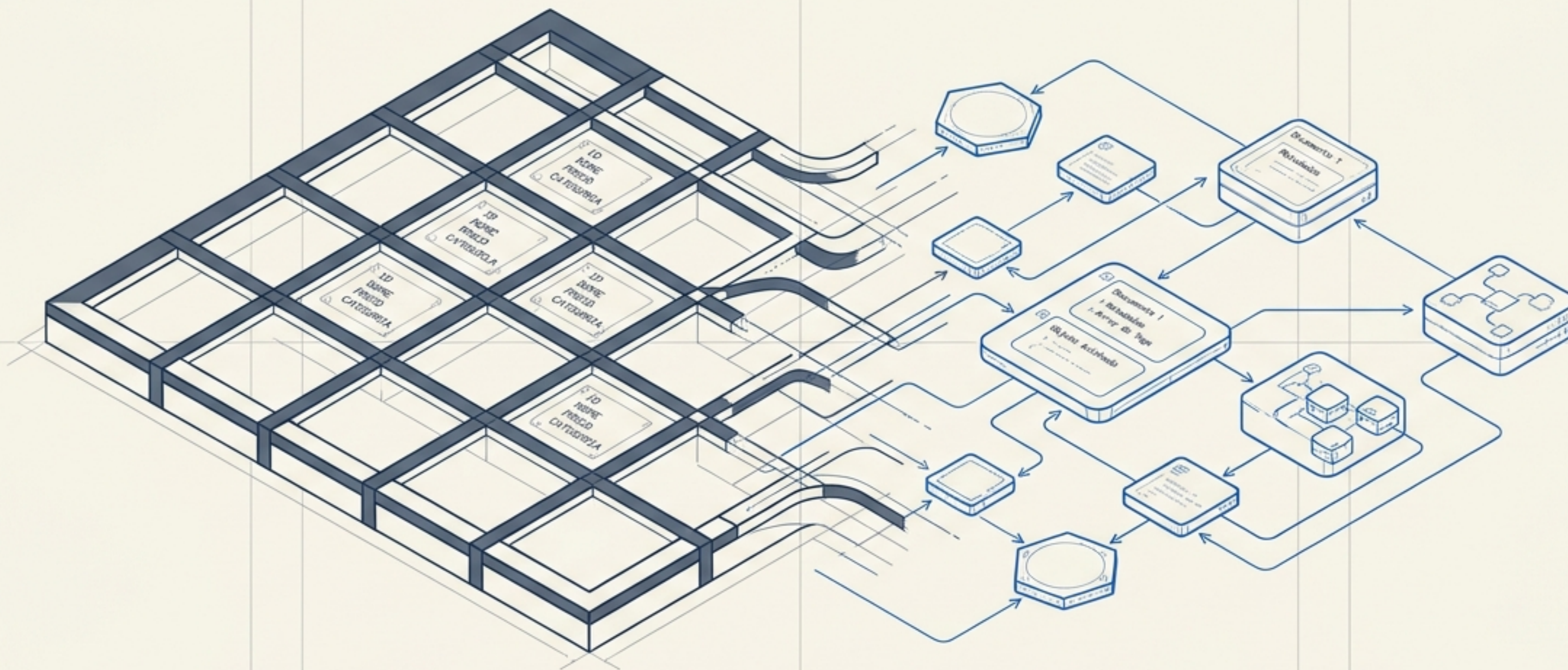




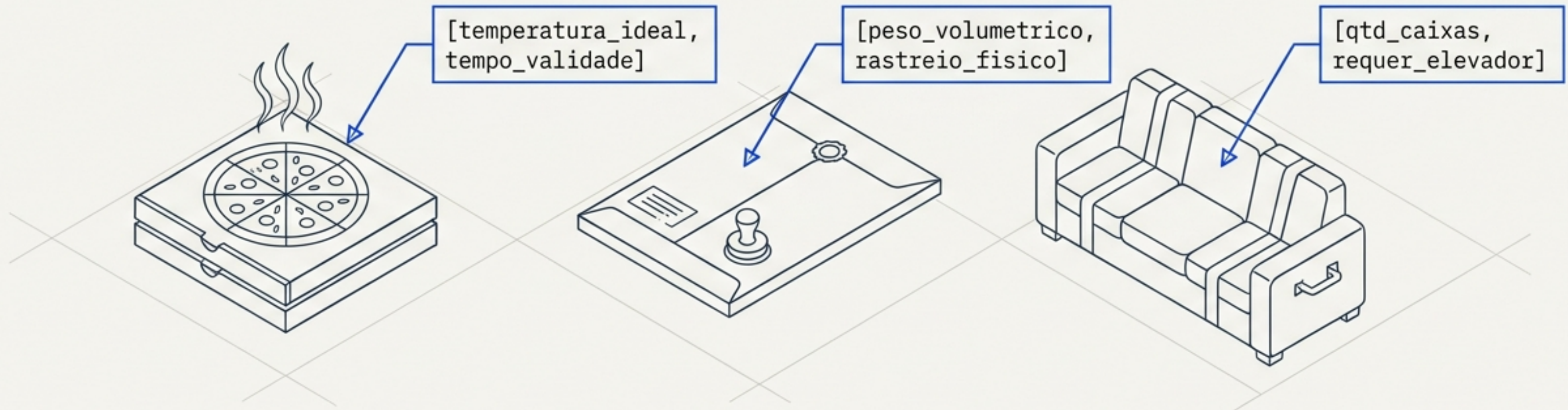
O Paradigma Documento e Catálogos Flexíveis

Dominando JSONB, Schema-on-Read e Buscas Otimizadas no PostgreSQL 16

Engenharia de Dados & Arquitetura de Software

0 Desafio da Alta Variância: Modelando o Caos

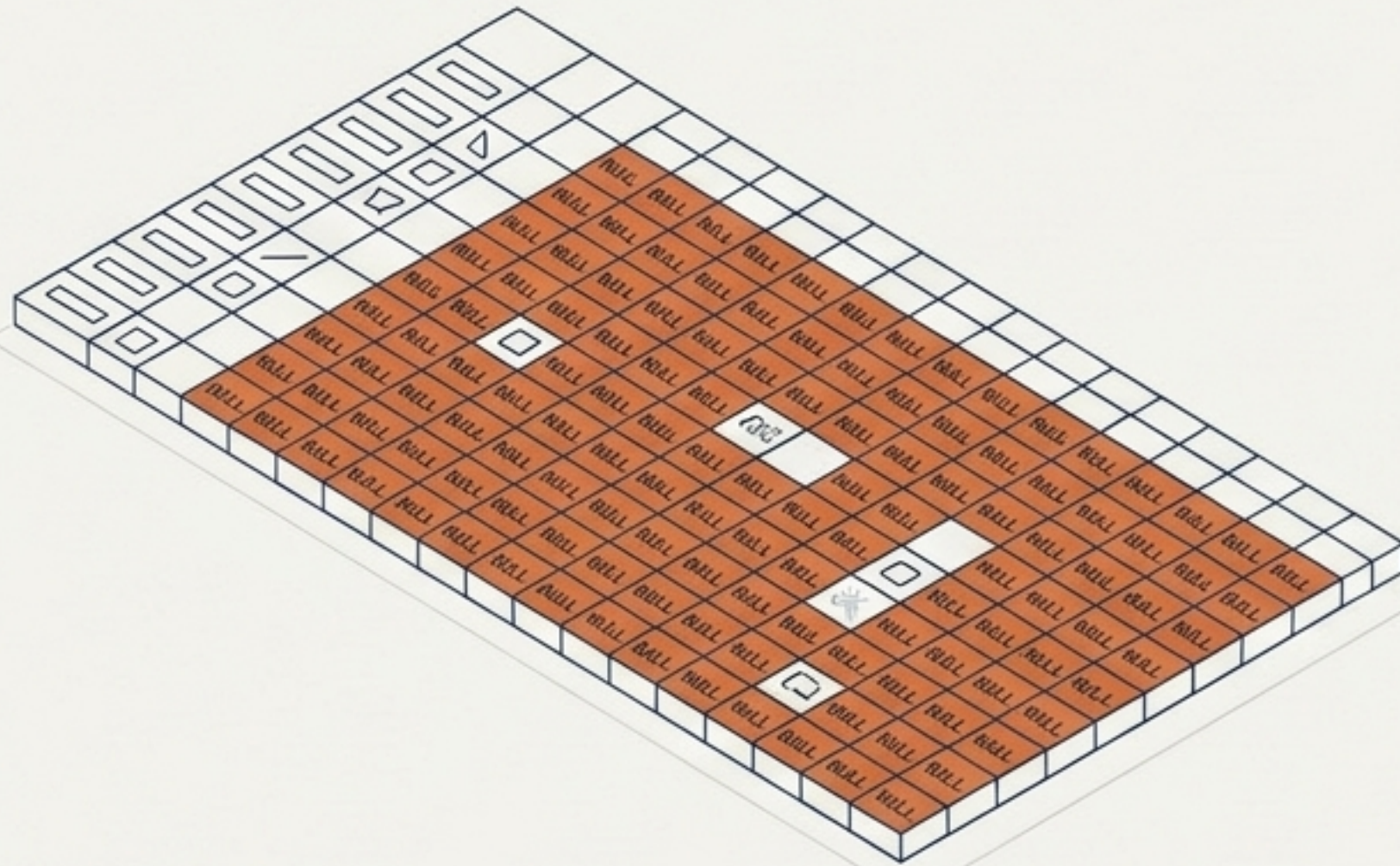
Como estruturar uma tabela `delivery_items` em uma operação logística que transporta absolutamente tudo, exigindo atributos únicos por categoria?



Atributos exclusivos para cada tipo de entidade quebram a previsibilidade e a eficiência de um esquema rígido.

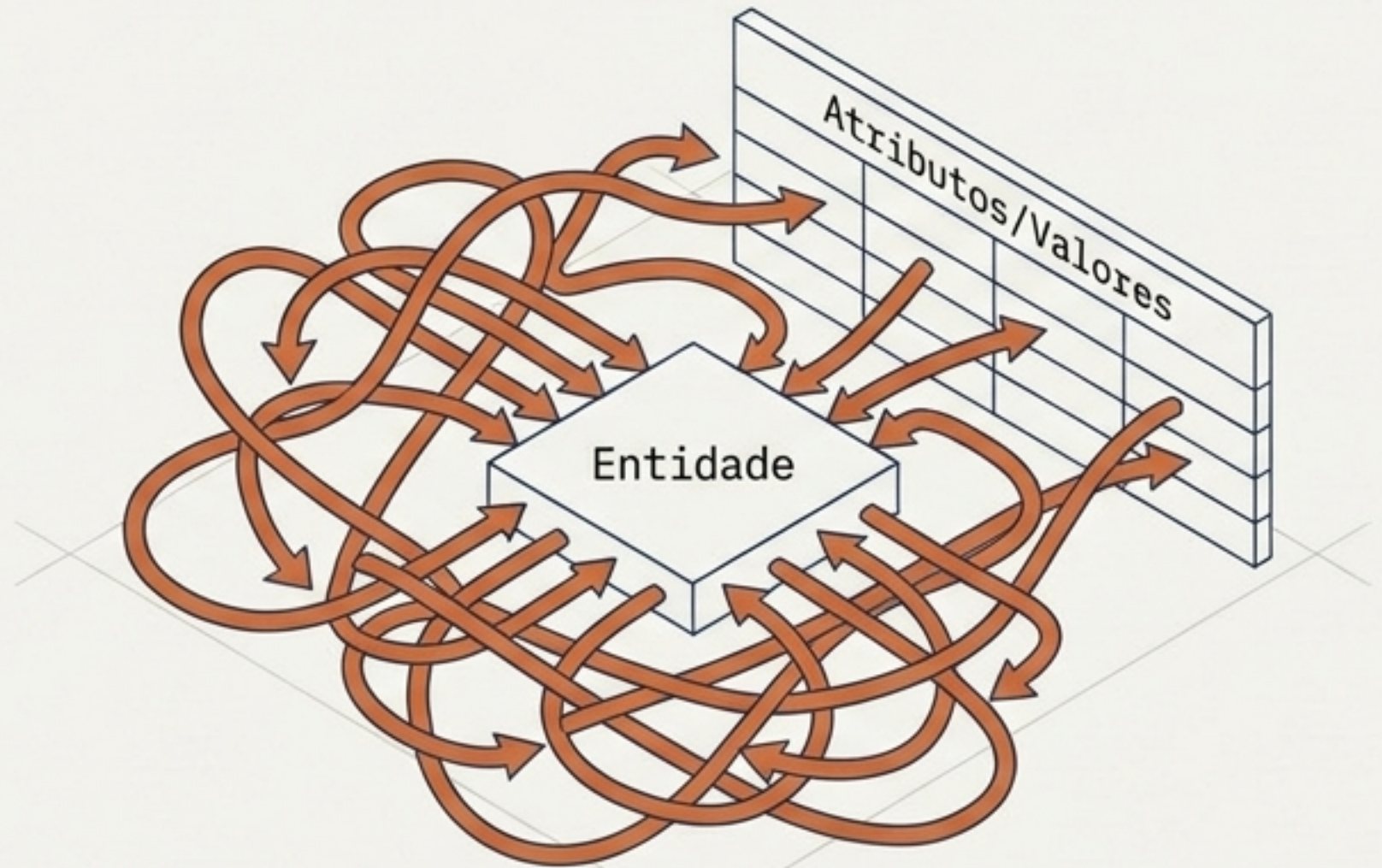
Tentativas Frustradas no Relacional Puro

Tabelas Largas (Wide Tables)



Requer mais de 50 colunas. 90% dos campos terminam como NULL. Causa desperdício massivo de disco e manutenção frágil.

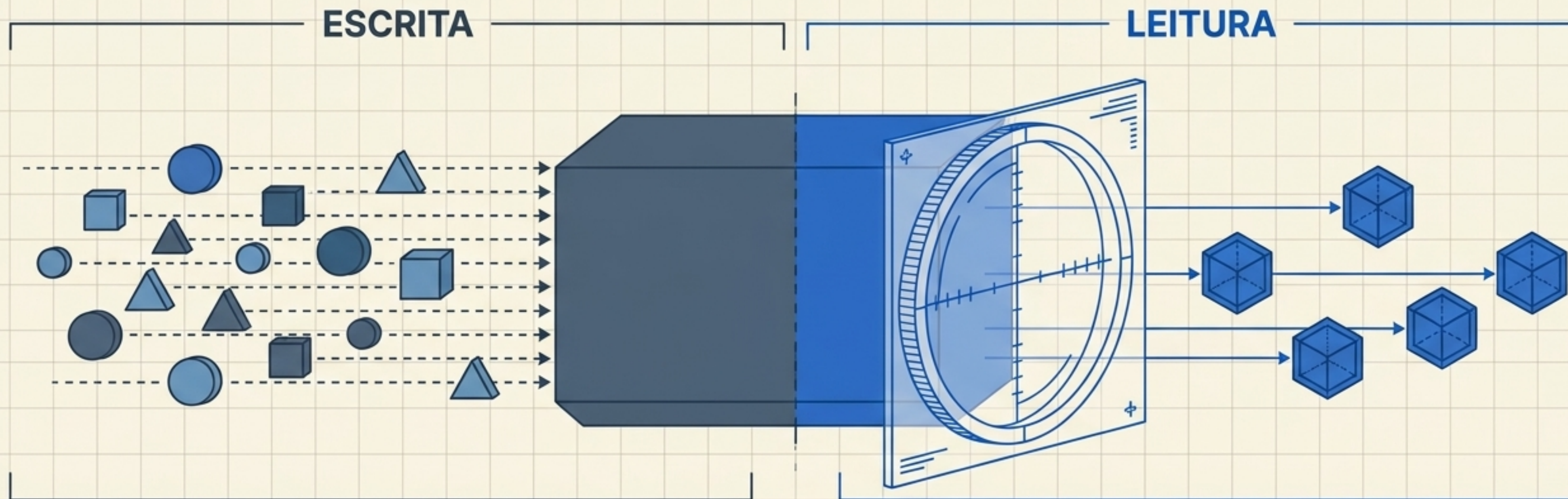
Padrão EAV (Entity-Attribute-Value)



Pesadelo absoluto de performance. Exige múltiplos JOINS massivos na mesma tabelas para reconstruir e ler uma única entidade.

A Inversão da Validação: Schema-on-Read

No paradigma de documento, a estrutura não é engessada no banco.



Gravação imediata. O banco aceita o payload sem impor um esquema rígido prévio.

O esquema é imposto dinamicamente pela aplicação ou pela query no momento da extração. Flexibilidade total, integridade mantida.

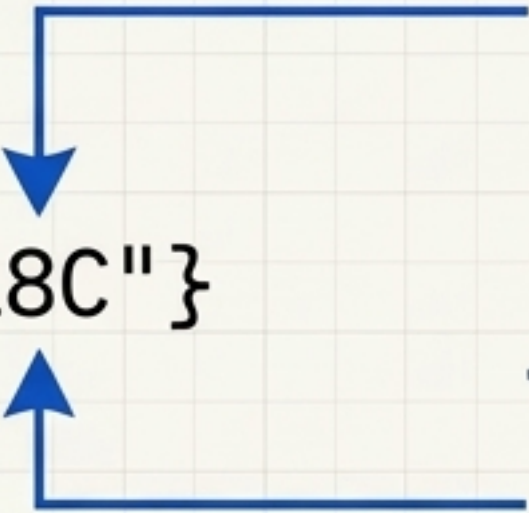
O Motor da Flexibilidade: Por que JSONB?

	JSON 	JSONB 
Formato de Armazenamento	Texto Puro	Binário Decomposto
Custo de Leitura	Reparsa a string inteira a cada query	Leitura direta e otimizada
Suporte a Índices	Extremamente Limitado	Suporte Total (GIN)

Regra de Ouro da Arquitetura:
No PostgreSQL, utilize sempre JSONB para modelagem de documentos.

Navegando no Documento: Extração de Dados.

```
{  
  "payload": {  
    "detalhes": {  
      "temperatura": "-18C"}  
    }  
  }  
}
```



-> Retorna Objeto JSON.

Permite o encadeamento contínuo na árvore.

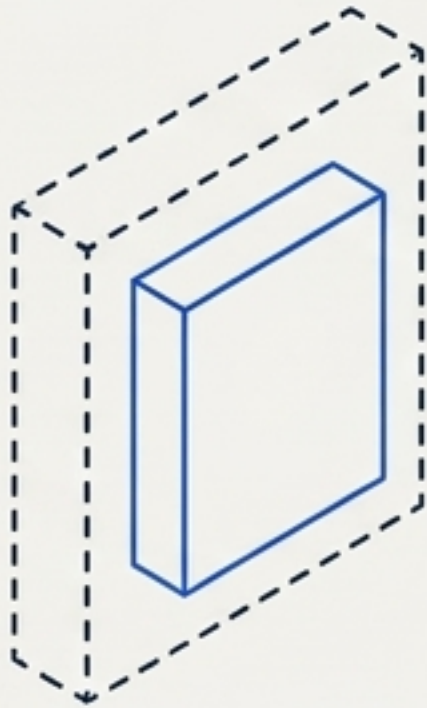
->> Retorna Texto Puro.

Finaliza a extração, ideal para projeção no SELECT ou comparações de valor.

```
SELECT payload -> 'detalhes' ->> 'temperatura' FROM delivery_items;
```

Interrogando o Documento: Operadores Nativos.

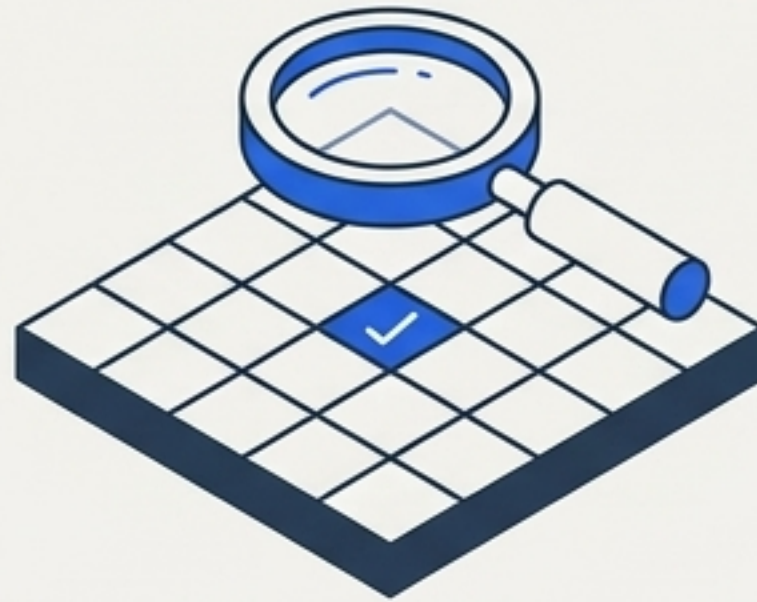
Contém (@>)



O documento à esquerda contém integralmente o documento à direita?

```
payload @> '{"tipo": "comida"}'
```

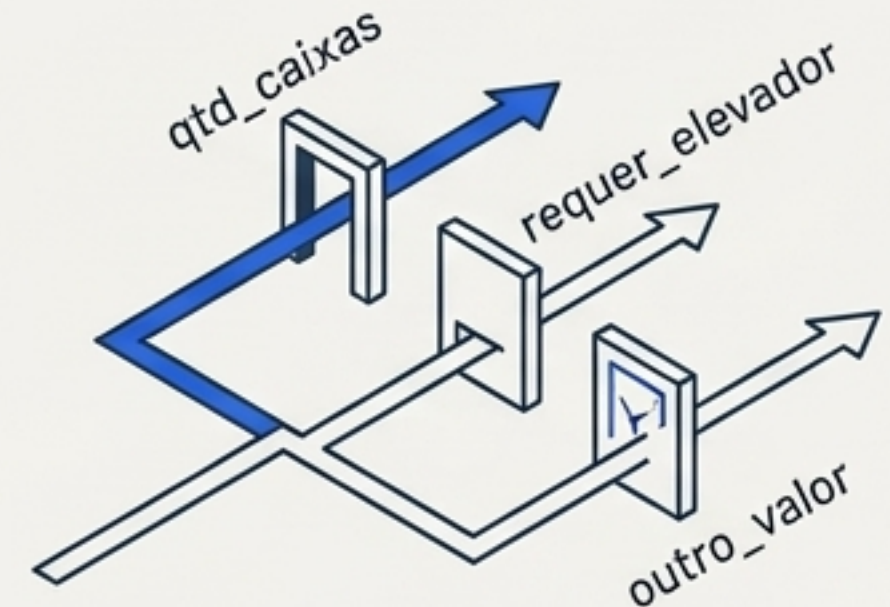
Existe a Chave? (?)



Verifica se uma chave específica existe no nível raiz do objeto.

```
payload ? 'rastreo_fisico'
```

Existe Alguma? (?|)



Operador lógico OR. Retorna verdadeiro se qualquer chave do array existir.

```
payload ?| array['qtd_caixas',  
                  'requer_elevador']
```

O Segredo da Escala: Índices GIN.

Buscas complexas com @> ou ? em milhões de linhas destruirão sua performance sem a indexação correta.

Sequential Scan - Custo Linear



GIN Scan - Custo Logarítmico (Milissegundos)

GIN Directory



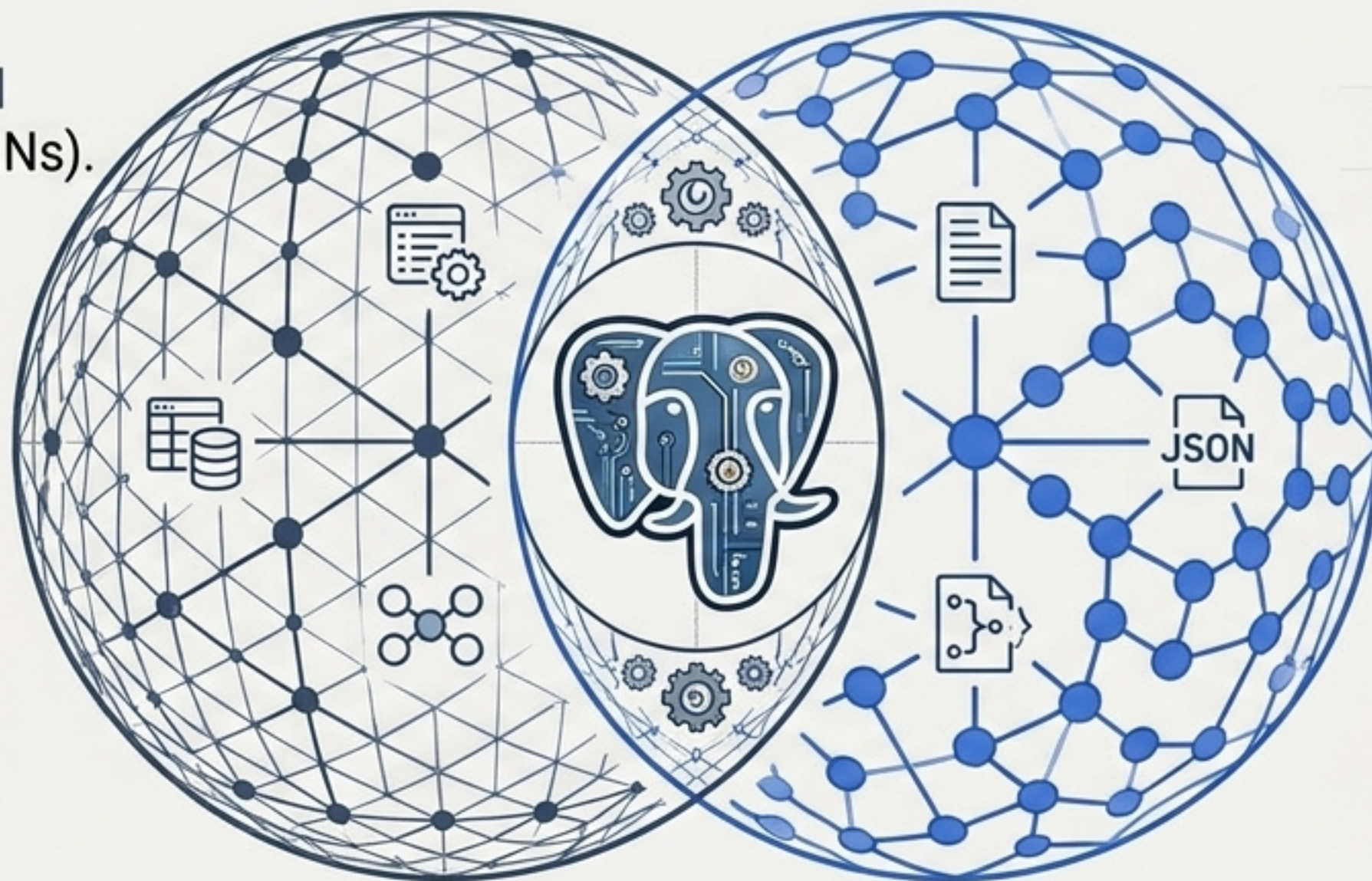
Concept Explanation:

O Índice GIN (Generalized Inverted Index) mapeia e inverte chaves/valores internos do JSONB, criando um diretório de acesso ultrarrápido para estruturas aninhadas.

```
CREATE INDEX idx_payload ON delivery_items USING GIN (payload);
```

O Fim do Falso Dilema: SQL vs NoSQL.

Integridade Relacional
(ACID, Constraints, JOINS).



Agilidade de Documento
(NoSQL, JSONB, Schema-on-Read).

O Postgres não é apenas um banco relacional clássico. Com o motor JSONB e índices GIN, ele é uma plataforma multi-modelo de alta performance. Você obtém a flexibilidade do NoSQL sem abrir mão das garantias ACID.

Próxima Etapa: 3 Horas de Hands-on Prático.